SOFTWARE ENGINEERING LAB – LAB 6

Code and Branch Coverage Analysis

Section H | Assigned repo: SETAPESU26/L6_L7_27_traffic_fine (Traffic Fine Calculator)
Student Name: Samvith Vaddiparthi | SRN: PES1UG24AM468 | Date: 12 November 2026

1. Objective

Measure how much of `Lab-6/traffic_fine.py` the tests execute, using Python's `coverage` tool with pytest. Then write test cases to reach at least 90% line (code) coverage and at least 90% branch coverage. **Both suites reach 100%.**

Function under test	Behaviour
`speeding_fine(speed, limit, zone)`	0 if not speeding; base fine by zone (school 2000, highway 1000, other 500), ×2 if over by more than 20, ×3 if over by more than 40
`license_action(points, is_commercial, repeat_offender)`	≥12 points: revoke (commercial) / suspend; 8–11: suspend (repeat offender) / warning; 4–7: notice; else none
`total_penalty(fines)`	Sums the fines, raises ValueError on a negative fine, caps the total at 10000

Environment: macOS, Python 3.12.4, pytest 9.2.0, coverage 7.17.2 (installed with `pip install coverage pytest` in a virtual environment).

2. Baseline (starter test file)

The starter `test_traffic_fine.py` has two tests (`test_no_speeding`, `test_clean_record`). They cover only **26% of the statements** (9 of 35) and **21% in branch mode**. Lines 4–15, 20–22, 24–26, 28 and 33–40 were never executed: every zone, every fine band, most licence actions and all of `total_penalty`. As the handout says, screenshots are only given for results at or above 90%.

3. Code (line) coverage – CodeCoverageTest_PES1UG24AM468_traffic_fine.py

Goal: every line of `traffic_fine.py` runs at least once. I chose one test for each line that was not yet executed:

Test	Call	Lines it executes
test_no_speeding (starter)	`speeding_fine(50, 60, "city")` → 0	2–3
test_school_zone_excess_over_40	`speeding_fine(105, 60, "school")` → 6000	4–6, 11–12
test_highway_zone_excess_over_20	`speeding_fine(130, 100, "highway")` → 2000	7–8, 13–14
test_city_zone_small_excess	`speeding_fine(70, 60, "city")` → 500	9–10, 15
test_clean_record (starter)	`license_action(2, False, False)` → none	19, 23, 27, 29
test_commercial_driver_high_points_revoked	`license_action(12, True, False)`	20–21

test_private_driver_high_points_suspended	`license_action(15, False, False)`	22
test_repeat_offender_medium_points_suspended	`license_action(9, False, True)`	24–25
test_first_offender_medium_points_warning	`license_action(8, False, False)`	26
test_low_points_notice	`license_action(5, False, False)`	28
test_total_penalty_capped_at_10000	`total_penalty([6000, 5000])` → 10000	33–35, 37–40
test_total_penalty_negative_fine_raises	`total_penalty([500, -100])` → ValueError	36

```
coverage run -m pytest CodeCoverageTest_PES1UG24AM468_traffic_fine.py
coverage report -m
```

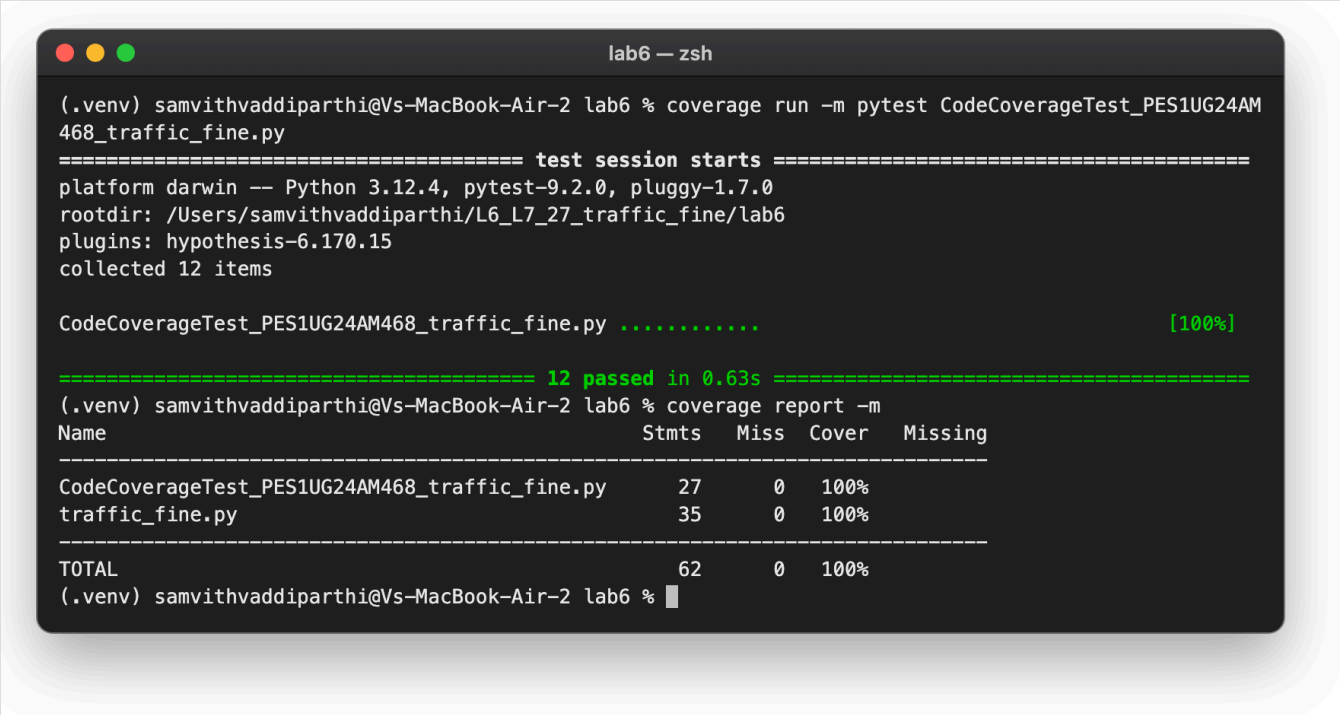


*Figure 1 – Code coverage: 12 tests pass, traffic_fine.py 35/35 statements, **100%** line coverage*

4. Analysis – 100% line coverage is not 100% branch coverage

Running the same 12 tests in branch mode shows that one decision outcome was never taken:

```
coverage run --branch -m pytest CodeCoverageTest_PES1UG24AM468_traffic_fine.py
coverage report -m
```

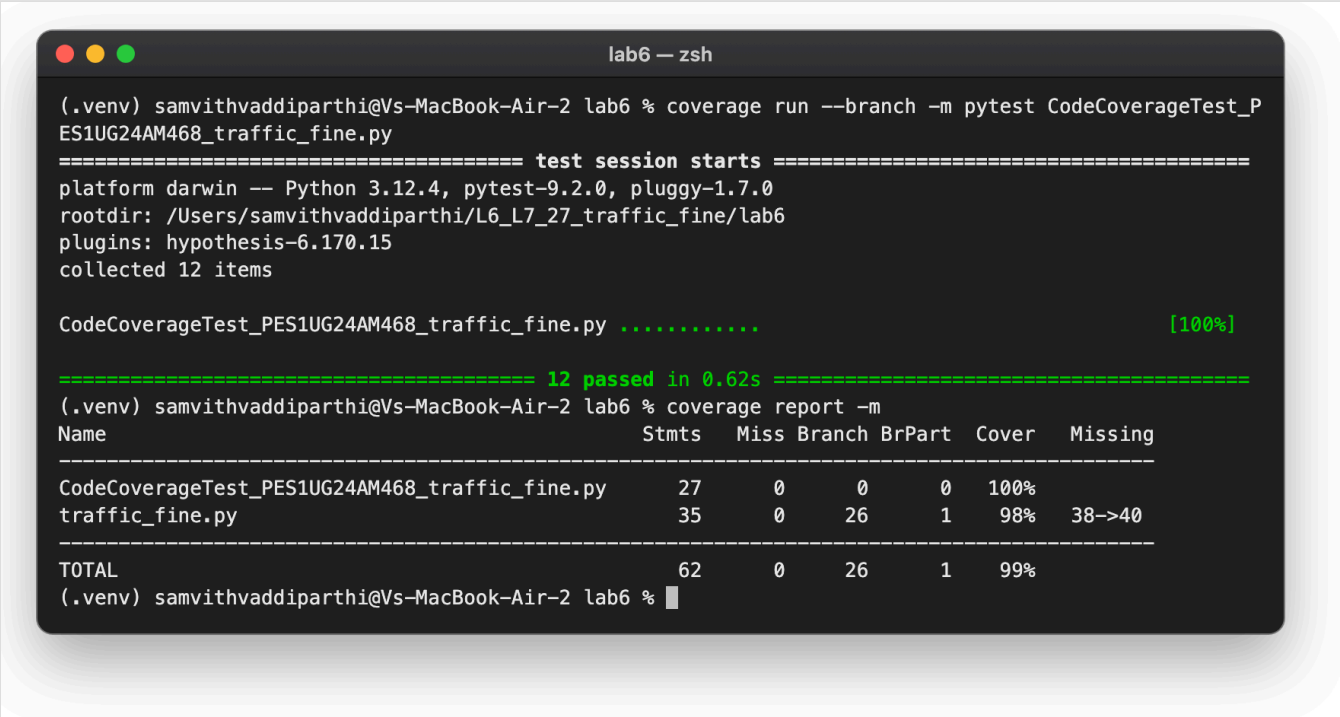


Figure 2 – The line-coverage suite in branch mode: 26 branches, 1 partial (`38->40`), 98%

`38->40` means "line 38 jumps straight to line 40", i.e. `if total > 10000` is **False**. The only non-raising `total_penalty` test summed to 11000, so the condition was always True. Every line still ran, but no test checked that a normal total (≤ 10000) is returned unchanged. This is the "false sense of security" the handout warns about. Line coverage also cannot show untested boundary values (e.g. `excess == 20` vs `21`), where off-by-one bugs usually hide.

5. Branch coverage – BranchCoverageTest_PES1UG24AM468_traffic_fine.py

Goal: every decision evaluates to both True and False, and every comparison is tested on both sides of its boundary. The suite uses `pytest.mark.parametrize` and has 50 test cases:

Function	Decisions	Cases (True and False sides + boundaries)
speeding_fine	`speed <= limit`, `zone == "school"`, `zone == "highway"`, `excess > 40`, `excess > 20`	speed below / equal to / 0 vs limit; 4 zones (school, highway, city, rural) × 6 excess values (1, 20, 21, 40, 41, 75) = 24 combinations; case-sensitive zone "School"
license_action	`points >= 12`, `is_commercial`, `points >= 8`, `repeat_offender`, `points >= 4`	13 cases at points 0, 2, 3, 4, 7, 8, 11, 12, 15, 20 with every flag combination that changes the result (e.g. commercial at 11 points → warning, not revoke)
total_penalty	`for fine in fines` (0 / 1+ iterations), `fine < 0`, `total > 10000`	empty list → 0; normal sum 3500 (**covers 38→40**); zero fines; exactly 10000 (not capped); 10001 → 10000; far above cap; negative first / later fine → ValueError; tuple input

```
coverage run --branch -m pytest BranchCoverageTest_PES1UG24AM468_traffic_fine.py
coverage report -m
```

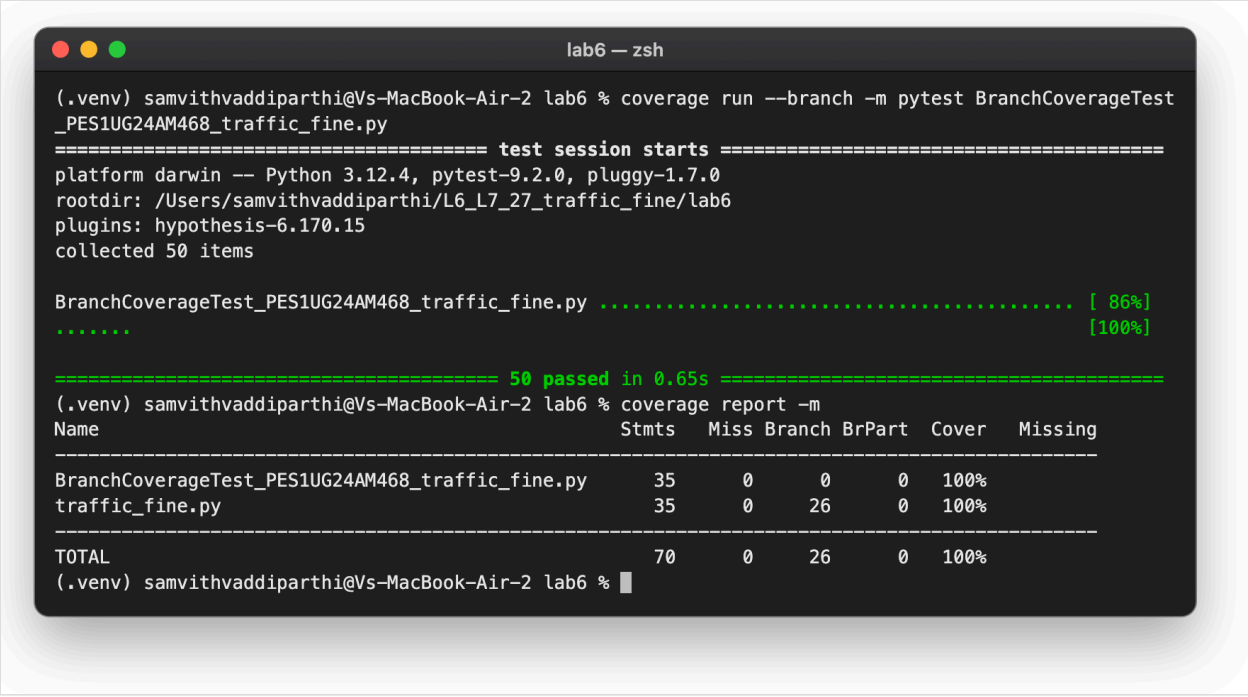


*Figure 3 – Branch coverage: 50 tests pass, traffic_fine.py 35/35 statements and 26/26 branches, **100%***

6. Results summary

Test suite	Tests	Statements covered	Branches covered	Coverage reported
Starter test_traffic_fine.py	2	9 / 35	– (4 partial)	**26% line, 21% branch**
CodeCoverageTest_PES1UG24AM468_traffic_fine.py	12	35 / 35	25 / 26 (38→40 missed)	**100% line (98% branch)**
BranchCoverageTest_PES1UG24AM468_traffic_fine.py	50	35 / 35	26 / 26	**100% branch**

7. Conclusion

- Line coverage went from 26% to **100%** with 10 targeted tests (one per unexecuted line).
- That suite still missed one branch (`38->40`). Branch coverage exposes untested decision outcomes that line coverage hides.
- Branch coverage reached **100%** by testing both outcomes of every decision and the values on each side of every boundary (20/21, 40/41, 3/4, 7/8, 11/12, 10000/10001, empty list).
- No test failed, so `traffic_fine.py` behaves as specified on all of these paths.

Deliverables (GitHub, 2 commits):
Commit 1 – `Lab-6/CodeCoverageTest_PES1UG24AM468_traffic_fine.py` + `Lab-6/screenshots/code_coverage_report.png`
Commit 2 – `Lab-6/BranchCoverageTest_PES1UG24AM468_traffic_fine.py` + `Lab-6/screenshots/branch_coverage_report.png`, `code_tests_under_branch_coverage.png`